Algorithm: QUANTUM ORDER FINDING

1. Initial state: $|0\rangle^{\otimes t}\ |1\rangle^{\otimes n}$
2. Create a superposition on the first register: $\frac{1}{\sqrt{2^t}} \sum_{j=0}^{2^t-1} |j\rangle\,|1\rangle$
3. Apply the sequence of controlled $U_a^{2^j}$ gates: $\frac{1}{\sqrt{r\,2^t}} \sum_{s=0}^{r-1} \sum_{j=0}^{2^t-1} e^{\frac{2\pi i s j}{r}} |j\rangle\,|u_s\rangle$
4. Apply inverse Fourier transform to the first register: $\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |2^t \frac{s}{r}\rangle\,|u_s\rangle$
5. Measure the first register ➜ estimate of $\frac{s}{r}$
6. Apply continued fractions algorithm ➜ order r

RUNTIME $O(n^2 \log n \log\log n)$ $\quad n = \lceil \log N \rceil$

Discussion

The order finding algorithm can fail

➡ QPE might produce a bad estimate for s/r

- This occurs with probability at most ε, thanks to the choice of t,

$$t = 2n + 1 + \lceil \log(2 + \frac{1}{2\varepsilon}) \rceil$$

- We can make this probability smaller, with a negligible increase in the size of the circuit

➡ If r and s have a common factor, the number r returned by the CFA is a factor of r, not the order r itself.

- r and s should be co-prime

Discussion

Let us estimate the probability that r and s are co-prime

- the number of integers $s < r$ and co-prime to r is given by the Eulero function $\phi(r) = \Omega\left(\frac{r}{\log\log r}\right)$
- thus, r and s are co-prime with probability

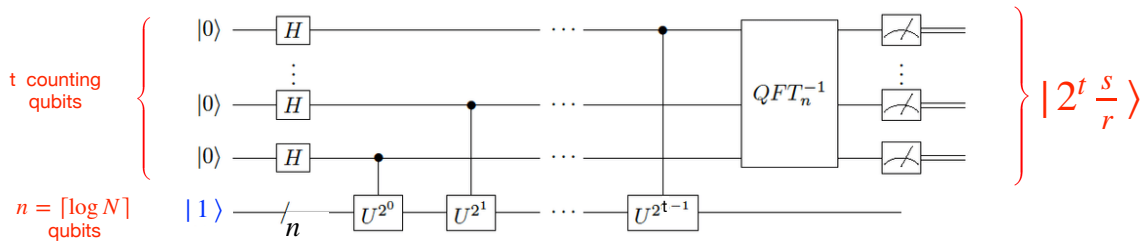
$$\Omega\left(\frac{1}{\log\log r}\right) = \Omega\left(\frac{1}{\log\log N}\right) = \Omega\left(\frac{1}{\log n}\right)$$

$r < N$ $\qquad$ $n = \lceil \log N \rceil$

An expected number of $O(\log n)$ repetitions of the algorithm allows to observe, with high probability, a phase s/r s.t. s and r are co-prime, and therefore CFA produces r

There are better methods, requiring O(1) repetitions

Example



$N = 21\,, \quad n = \lceil \log_2 N \rceil = 5\,, \quad a = 11\,, \quad GCD(N, a) = 1\,, \quad r = 6$

We choose $t = 2n + 1 + \lceil \log(2 + \frac{1}{2\varepsilon}) \rceil$ with $\varepsilon = 1/4$

$$t = 2 \cdot 5 + 1 + 2 = 13$$

This choice guarantees that for each s, we obtain an estimate of the phase s/r accurate to 11 bits, with probability at least $3/4$

When we measure the first register, the value x will be close to a multiple s of

$$\frac{2^t}{r} = \frac{2^{13}}{6}$$

where $0 \leq s < r$

Example

- Suppose we measure $x = 5461$
- $\varphi = 5461/2^{13} = 5461/8192$ is an approximation of s/r with $r < 21$
- We run CFA

$$\frac{5461}{8192} \equiv [0, 1, 1, 1, 2730] = 0 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{1}{2730}}}$$

- The convergents are

$$0, 1, \frac{1}{2}, \frac{2}{3}, \frac{5461}{8192}$$

- Thus, we compute the fraction $2/3$, which is the **unique** satisfying the requirements

$$\left|\varphi - \frac{s}{r}\right| \leq \frac{1}{2N^2} \quad \text{and} \quad r < N$$

- 3 is the candidate for the order, thus we check whether $a^3 \mod N = 1$
- $a^3 \mod N = 11^3 \mod 21 = 8$ 

Example

- Now, suppose we measure $x = 1365$
- $\varphi = 1365/2^{13} = 1365/8192$ is an approximation of s/r with $r < 21$
- We run CFA

$$\frac{1365}{8192} \equiv [0, 6, 682, 2] = 0 + \cfrac{1}{6 + \cfrac{1}{682 + \frac{1}{2}}}$$

- The convergents are

$$0, \frac{1}{6}, \frac{682}{4093}, \frac{1365}{8192}$$

- Thus, we compute the fraction $1/6$
- 6 is the candidate for the order, thus we check whether $a^6 \mod N = 1$
- $a^6 \mod N = 11^6 \mod 21 = 1$

SHOR's factoring algorithm

REDUCTION FROM FACTORING TO ORDER-FINDING (Miller, 1975)

Assume that

- N is composite (we can test primality efficiently)
- N is odd
- N is not a prime power

If $N = p^k$, p prime, we can factor N in polynomial time

Suppose $N = p^k$

Then $k \leq \log_2 N$

Compute each of the first $\log_2 N$ roots of N, and test if the resulting number is a prime integer

Reduction from factoring to order finding

Let $a < N$ be s.t. $GCD(a, N) = 1$

$\rightsquigarrow$ if $GCD(a, N) > 1$, we have found a factor of N

Let $r = \text{ord}\,(a)$

$$a^r \bmod N = 1$$

$$(a^r - 1) \bmod N = 0$$

If r is even, this factorises

$$(a^{r/2} - 1)(a^{r/2} + 1) \bmod N = 0$$

The two terms are both positive, thus we have 4 possible cases

$$(a^{r/2}-1)(a^{r/2}+1) \bmod N = 0$$

(1) $a^{r/2} \bmod N = 1$

but this **cannot occur** as r is the least integer satisfying $a^r \bmod N = 1$

(2) $a^{r/2} \bmod N = -1 \quad (a^{r/2} \bmod N = N-1)$

the **reduction fails**, we need to start over again, choosing a different a

(3) $(a^{r/2}-1)(a^{r/2}+1) = N$

$(a^{r/2}-1)$ and $(a^{r/2}+1)$ are factors of N, that we output

$$(a^{r/2}-1)(a^{r/2}+1) \bmod N = 0$$

(4) $(a^{r/2}-1)(a^{r/2}+1) = kN, \qquad k \geq 2$

$N \mid (a^{r/2}-1)(a^{r/2}+1)$

- any factor of N, is also a factor of either $(a^{r/2}-1)$ or $(a^{r/2}+1)$, or both
- the common factor cannot be N, since

$$(a^{r/2} \pm 1) \bmod N \neq 0$$

indeed, $a^{r/2} \bmod N \neq 1$ (by definition of order), and we excluded that $a^{r/2} \bmod N = -1$

Using Euclid's algorithm, we can compute $GCD(a^{r/2}-1, N)$ and $GCD(a^{r/2}+1, N)$ in time polynomial in $\log N$, and find a non trivial factor of N

Example

$(a^{r/2} - 1)(a^{r/2} + 1) \bmod N = 0$

→ $N = 21 \qquad a = 6 \qquad GCD(N, a) = 3$

3 is a factor of $N = 21$

→ $N = 15 \qquad a = 2 \qquad r = 4$

$(a^{r/2} - 1)(a^{r/2} + 1) = N$

$(2^2 - 1)(2^2 + 1) = 3 * 5$ case (3)

→ $N = 33 \qquad a = 2 \qquad r = 10$

$a^{r/2} \bmod N = 2^5 \bmod 33 = 32 \bmod 33 = -1$ case (2)

$(a^{r/2} - 1) \bmod N = (2^5 + 1) \bmod 33 = 0$

The **reduction fails**

Example

$(a^{r/2} - 1)(a^{r/2} + 1) \bmod N = 0$

→ $N = 21 \qquad a = 11 \qquad GCD(21, 11) = 1 \qquad r = 6, \quad r$ is even

$a^{r/2} \bmod N = 11^3 \bmod 21 = 8 \neq -1$

$(a^{r/2} - 1)(a^{r/2} + 1) = k\,N, \quad k \geq 2$ case (4)

$GCD(a^{r/2} - 1, N) = GCD(N, (a^{r/2} - 1) \bmod N) = GCD(21,7) = 7$

$GCD(a^{r/2} + 1, N) = GCD(N, (a^{r/2} + 1) \bmod N) = GCD(21,9) = 3$

$N = 7 * 3$

→ $N = 21 \qquad a = 4 \qquad GCD(21, 4) = 1 \qquad r = 3, \quad r$ is odd

The **reduction fails**

Shor's algorithm

Input: a positive integer N, with $n = \lceil \log N \rceil$ bits

Output: a factor p of N

(1) IF N IS EVEN, OUTPUT 2 AND STOP

(2) USE A POLYNOMIAL TIME ALGORITHM TO DETERMINE IF N IS PRIME, OR A POWER OF A PRIME
IF THIS IS THE CASE, DECLARE IT AND STOP

(3) RANDOMLY CHOOSE AN INTEGER a such that $1 < a < N$
AND RUN EUCLID'S ALGORITHM TO DETERMINE $GCD(a, N)$
IF $GCD(a, N) > 1$, RETURN IT AND STOP

Shor's algorithm

(4) IF a AND N ARE CO-PRIME, USE A **QUANTUM CIRCUIT** TO FIND THE ORDER r OF a, modulo N

(5) IF r IS EVEN AND $a^{r/2} \mod N \neq -1$, COMPUTE $GCD(a^{r/2} - 1, N)$ AND $GCD(a^{r/2} + 1, N)$ AND RETURN AT LEAST ONE NON TRIVIAL FACTOR OF N

(6) OTHERWISE (r is odd, or $a^{r/2} \mod N = -1$) RETURN TO STEP (3) AND CHOOSE ANOTHER a

Shor's algorithm: discussion

- In summary, Shor's algorithm has two main parts:
  - a reduction of the factoring problem to the problem of order-finding, which can be done on a **classical computer**
  - a **quantum algorithm** to solve the order-finding problem (step (4))
- The algorithm returns **with high probability** a non trivial factor of any composite N
- All steps can be performed efficiently on a classical computer, EXCEPT (as far as we know) the order-finding step (step (4))
- By repeating the procedure, we may find a complete prime factorization of N

Success Probability

A single run only returns a factor of N with a certain probability, since the reduction to the order-finding problem **does not work** if

$$r \equiv 1 \mod 2 \quad \text{or} \quad a^{r/2} \equiv -1 \mod N$$

THEOREM

Let $N = \prod_{j=1}^{k} p_j^{n_j}$ be the factorization of N ($p_j \neq p_j$ for $i \neq j$)

If a is a random element in $\mathbb{Z}_N^*$, co-prime with N, and $r = \text{ord}(a)$, then

$$Prob\,(r = 0 \mod 2 \text{ AND } a^{r/2} \not\equiv -1 \mod N) \geq 1 - \frac{1}{2^k}$$

For semiprime integers ($k = 2$), the probability is at least 3/4 (RSA)

Shor's algorithm

The algorithm only needs to be repeated a constant number of times to successfully factor N with a high probability of success
(the expected number of iterations needed to find a factor does not grow with N)

↳ Shor's algorithm can factor N in a number of operations polynomial in the dimension of N (i.e., the number of bits of N)

↳ *exponential speed-up compared to the best classical algorithm*

Complexity: a closer look

The algorithm is polynomial in $n = \lceil \log_2 N \rceil$

It takes $O(n^2 \log n \log\log n)$ quantum gates using fast multiplication and the square-and-multiply algorithm, otherwise $O(n^3)$

The **most expensive operation** performed is **modular exponentiation**

- it is by far slower than the quantum Fourier transform and classical pre-/post-processing
- it dominates the overall cost of Shor's algorithms

The **constant factors** hidden by the asymptotic notation **remain substantial**

- these constant factors must be overcome, by heavy optimization, to make the algorithm practical
- current quantum computers are far from being capable of executing Shor's algorithm for cryptographically relevant problem sizes

Complexity: resource estimation

Craig Gidney, Martin Ekerå
How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits
Quantum 5: 433 (2021)

Estimate of the approximate cost of implementing Shor's algorithms

- in terms of **logical qubits** and **gates** in the abstract circuit model, where overheads from routing and error correction are not considered
- in terms of **runtime** and **physical qubit** usage in an error corrected implementation under plausible physical assumptions

Complexity: resource estimation

Craig Gidney, Martin Ekerå
How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits
Quantum 5: 433 (2021)

A **logical qubit** is an abstract, perfect stable qubit that performs as specified in a quantum algorithm *(it behaves exactly as we expect)*

A **physical qubit** is a physical device that behaves as a two-state quantum system, i.e., **an actual quantum implementation of a logical qubit** *(unstable entity)*

Most technologies used to implement qubits face issues of stability, decoherence, fault tolerance and scalability

↳ **logical qubits consist of many physical qubits** to provide stability, error-correction and fault tolerance needed to perform useful computations (➔ *quantum error correction*)

Complexity: resource estimation

Craig Gidney, Martin Ekerå
How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits
Quantum 5: 433 (2021)

The estimate of the number of physical qubits necessary to create a logical one depends on the quantum technology for coding physical qubits

- a reasonably **fault-tolerant logical qubit** that can be used effectively may need 10^3 to 10^4 physical qubits
- the study assumes that **3600 physical qubits per logical qubit** are needed to give a sufficiently low logical error rate to successfully execute Shor's algorithm

Complexity: resource estimation

Craig Gidney, Martin Ekerå
How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits
Quantum 5: 433 (2021)

ABSTRACT MODEL (n-bit integers)

Logical qubits $3n + 0.002\, n \log n$

Toffoli + T Gates $0{,}3\, n^3 + 0.0005\, n^3 \log n$

RSA Bits	Qubits	Toffoli + T Gates (billions)
1024	3092	0.4
2048	6189	2.7
3072	9287	9.9

Complexity: resource estimation

Craig Gidney, Martin Ekerå
How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits
Quantum 5: 433 (2021)

ERROR CORRECTED IMPLEMENTATION (2048-bit integers)

Physical qubits	20 millions
Expected runtime	0.31 days

This estimate implies that factoring a 2048 bit integer will take about **8 hours**, assuming only one run of the quantum part of the algorithm is needed

Distinguishing between physical and logical qubits is important

↳ to break RSA cryptography (factorize 2048-bit integers) we would need thousands of logical qubits, but millions of physical qubits

Discussion (from Scott Aaronson's lecture notes)

Shor's algorithm also gives exponential speedup for the **Discrete Log Problem** (as Shor showed in his original paper)

Given a prime p, an integer g, and an integer a, *find* x such that

$$g^x \bmod p = a$$

↳ Shor's algorithm breaks the **Diffie-Hellman Cryptosystem**

Shor's algorithm can also be modified to solve in polynomial time the **Discrete Log Problem on Elliptic Curve**

↳ Shor's algorithm breaks the **Elliptic Curve Cryptosystems**

It was noted shortly afterward that Shor's algorithm can be modified to solve *any* problem related to finding *hidden structures in abelian groups*

Almost all the public-key cryptosystems currently in use involve finding such hidden structures

A (very hard) question:

"To what extent can we generalize Shor's algorithm to solve problems about non-abelian groups?"

If Shor's algorithm could be generalized to handle them, it could be applied to solve efficiently other problems

Graph Isomorphism Problem

Given two graphs, decide whether they're isomorphic

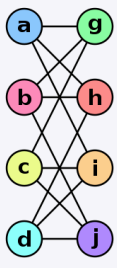
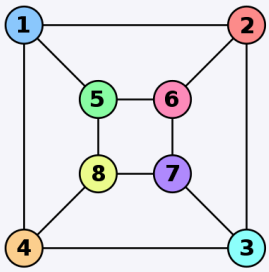
- It's a problem that no one yet knows how to solve in polynomial time, but that seems to have *too much structure* to be NP-complete
- Many people suspect that Graph Isomorphism is in P (the problem is easy in practice almost all of the time)
- It's a major unsolved problem in computer science

Graph Isomorphism

In graph theory, an **isomorphism of graphs** G and H is a bijection between the vertex sets of G and H

$$f : V(G) \to V(H)$$

such that any two vertices u and v of G are adjacent in G if and only if $f(u)$ and $f(v)$ are adjacent in H

Graph G	Graph H	An isomorphism between G and H
		*f*(*a*) = 1 *f*(*b*) = 6 *f*(*c*) = 8 *f*(*d*) = 3 *f*(*g*) = 5 *f*(*h*) = 2 *f*(*i*) = 4 *f*(*j*) = 7

These two graphs are isomorphic, despite their different looking drawings

Graph Isomorphism

In 2016, László Babai found a classical algorithm to solve Graph Isomorphism in *quasipolynomial* time $n^{polylog(n)}$

A generalization of Shor's algorithms to a situation where the underlying group is the **symmetric group** S_n, instead of an abelian group, would solve Graph Isomorphism in quantum polynomial time.

Symmetric group S_n over a finite set X with n elements is the group of all permutations from X to X, whose group operation is function composition

Post-quantum Cryptography

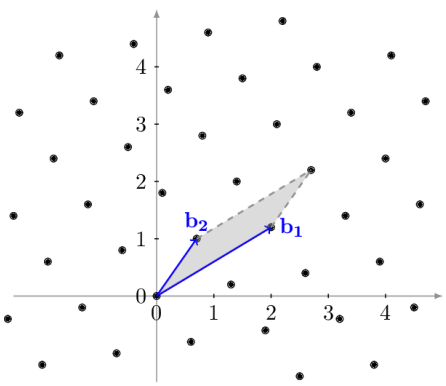
Shor's algorithm has stimulated research

- on the design and construction of quantum computers and new quantum algorithms
- on new cryptosystems that are secure from quantum computers

Public-key cryptosystems based on lattices

Given a collection of vectors $\{z_1, z_2, \ldots, z_n\}$ in $\mathbb{R}^n$, the lattice spanned by the collection is the set of all **integer linear combinations** of the vectors

$$L = \{a_1 z_1 + a_1 z_1 + \ldots + a_n z_n \mid \ a_1, a_2, \ldots, a_n \in \mathbb{Z}\}$$



Post-quantum Cryptography

Public-key cryptosystems have been developed around problems as

"Given $\{z_1, z_2, \ldots, z_n\}$, find the shortest non zero vector in L, or at least, a vector that's within a $\sqrt{n}$ factor of being the shortest"

We could break lattice-based cryptosystems if we could generalize Shor's algorithm to work for a non-abelian group called the *dihedral group* (Regev, 2005)

➔ *No one has yet succeeded in doing so*

Lattice-based cryptography is an alternative to RSA and Diffie-Hellman

NIST Post-Quantum Cryptography Standardization Process

- Announced in **2016** to protect data traffic against quantum computer attacks in the future
- A total of **69** international teams from the cryptography community have submitted proposals (signature and encryption/KEM schemes) by the deadline at the end of 2017
- After several rounds, NIST has now decided to standardize **four** of these procedures (July 2022)

Public-key Encryption and Key-establishment Algorithms

CRYSTALS-KYBER (lattice)

Digital Signature Algorithms

CRYSTALS-DILITHIUM (lattice)

FALCON (lattice)

SPHINCS+ (Hash-based)

Appendix
Theory of continued fractions

Theory of continued fractions

The idea of continued fractions is to approximate real numbers using finite number of integers

Let $a_0, a_1, a_2, a_3, \ldots \in \mathbb{N}$

We denote with $[a_0, a_1, a_2, a_3, \ \ldots\ ]$ the **continued fraction** defined as

$$[a_0, a_1, a_2, a_3, \ \ldots\ ] \equiv a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \frac{1}{a_3 + \ldots}}}$$

Using a continued fraction expansion, every real number x can be expressed in terms of integer numbers, and approximated with a rational

Example

Let us approximate π to the first 4 decimal places with a rational number.

$$\pi = 3{,}1415\ldots \quad = 3 + \frac{1415}{10000} \quad = 3 + \frac{1}{\frac{10000}{1415}}$$

$10000 = 7 * 1415 + 95$

$$= 3 + \frac{1}{7 + \frac{95}{1415}} \quad = 3 + \frac{1}{7 + \frac{1}{\frac{1415}{95}}} \quad = 3 + \frac{1}{7 + \frac{1}{14 + \frac{85}{95}}}$$

$1415 = 14 * 95 + 85$

$$= 3 + \frac{1}{7 + \frac{1}{14 + \frac{1}{\frac{95}{85}}}} \quad = 3 + \frac{1}{7 + \frac{1}{14 + \frac{1}{1 + \frac{10}{85}}}}$$

$95 = 85 + 10$

We consider this term small enough to be discarded

$$\approx 3 + \frac{1}{7 + \frac{1}{14 + 1}} = 3 + \frac{1}{7 + \frac{1}{15}} = \frac{333}{106}$$

$$CF_2(\pi) = [3, 7, 15] = \frac{333}{106} = 3.1415094339$$

Theory of continued fractions

- A continued fraction is **finite** if it is defined by a finite set of elements $a_0, a_1, \ldots, a_k \in \mathbb{N}$
- A number x has a **finite** continued fractions expansion if and only if $x \in \mathbb{Q}$

$$\frac{427}{512} \equiv [0, 1, 5, 42, 2] = 0 + \frac{1}{1 + \frac{1}{5 + \frac{1}{42 + \frac{1}{2}}}}$$

$$\frac{1365}{8192} \equiv [0, 6, 682, 2] = 0 + \frac{1}{6 + \frac{1}{682 + \frac{1}{2}}}$$

Example

Let us approximate the rational number 427/512

$$\frac{427}{512} = 0 + \frac{1}{\frac{512}{427}} = 0 + \frac{1}{1 + \frac{85}{427}} = 0 + \frac{1}{1 + \frac{1}{\frac{427}{85}}}$$

$$= 0 + \frac{1}{1 + \frac{1}{5 + \frac{2}{85}}} = 0 + \frac{1}{1 + \frac{1}{5 + \frac{1}{\frac{85}{2}}}}$$

$$= 0 + \frac{1}{1 + \frac{1}{5 + \frac{1}{42 + \frac{1}{2}}}} = [0, 1, 5, 42, 2]$$

$$512 = 427 + 85$$
$$427 = 85 * 5 + 2$$
$$85 = 42 * 2 + 1$$

The fractional part 1/2 is the reciprocal of 2, so its approximation in this scheme is exact

EXERCISE

- Verify that $\frac{415}{93} = [4, 2, 6, 7]$
- Verify that $\varphi = \frac{1 + \sqrt{5}}{2} \approx 1{,}6180339887\ldots = [1, 1, 1, 1, ...]$

Theory of continued fractions

If x is an irrational number, then its **continued fractions representation continues indefinitely** and produces a sequence of rational approximations that converge to the starting number x as a limit

- $\sqrt{19} = [4, 2, 1, 3, 1, 2, 8, 2, 1, 3, 1, 2, 8, \ldots]$
  The pattern repeats indefinitely with a period of 6
  Sequence A010124 in the OEIS (On-Line Encyclopedia of Integer Sequences)
- $e = [2, 1, 2, 1, 1, 4, 1, 1, 6, 1, 1, 8, \ldots]$
  The pattern repeats indefinitely with a period of 3 except that 2 is added to one of the terms in each cycle
  Sequence A003417 in the OEIS
- $\varphi = [1, 1, 1, 1, 1, 1, 1, 1, \ldots]$
  The golden ratio, sequence A000012 in the OEIS
- $\pi = [3, 7, 15, 1, 292, 1, 1, 1, 2, 1, 3, 1, \ldots]$
  No pattern has ever been found in this representation
  Sequence A001203 in the OEIS

Theory of continued fractions: remarks

Let x be an irrational number

The finite continued fractions obtained truncating the infinite continued fraction representation for x provide rational approximations to the number x.

These rational numbers are called the convergents of the continued fraction

Definition

Let $CF(x) = [a_0, a_1, a_2, \ldots]$, $a_i \in \mathbb{N}^+$, be the continued fraction representation of $x \in \mathbb{R}$

The k**-th convergent** of x is the rational number

$$CF_k(x) = [a_0, a_1, \ldots a_k] = p_k \,/\, q_k$$

Lemma

$CF_k(x) = [a_0, a_1, \ldots a_k] = p_k \,/\, q_k$ is the best rational approximation of x with denominator at most q_k

↳ p_k/q_k is closer to x than any approximation with a smaller or equal denominator

Example

$$CF_k(x) = [a_0, a_1, \ldots a_k] = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\ldots + \frac{1}{a_k}}}} = \frac{p_k}{q_k}$$

$$\frac{427}{512} \equiv [0, 1, 5, 42, 2] = 0 + \cfrac{1}{1 + \cfrac{1}{5 + \cfrac{1}{42 + \frac{1}{2}}}}$$

The convergents are

- $\frac{p_0}{q_0} = 0$
- $\frac{p_1}{q_1} = 0 + \frac{1}{1} = 1$
- $\frac{p_2}{q_2} = 0 + \cfrac{1}{1 + \frac{1}{5}} = \frac{5}{6}$
- $\frac{p_3}{q_3} = 0 + \cfrac{1}{1 + \cfrac{1}{5 + \frac{1}{42}}} = \frac{211}{253}$
- $\frac{p_4}{q_4} = 0 + \cfrac{1}{1 + \cfrac{1}{5 + \cfrac{1}{42 + \frac{1}{2}}}} = \frac{427}{512}$

Example

$$\frac{1365}{8192} \equiv [0, 6, 682, 2] = 0 + \cfrac{1}{6 + \cfrac{1}{682 + \frac{1}{2}}}$$

The convergents are

- $\frac{p_0}{q_0} = 0$
- $\frac{p_1}{q_1} = 0 + \frac{1}{6} = \frac{1}{6}$
- $\frac{p_2}{q_2} = 0 + \cfrac{1}{6 + \frac{1}{682}} = \frac{682}{4093}$
- $\frac{p_3}{q_3} = 0 + \cfrac{1}{6 + \cfrac{1}{682 + \frac{1}{2}}} = \frac{1365}{8192}$

Example

$$\frac{5461}{8192} \equiv [0, 1, 1, 1, 2730] = 0 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{1}{1 + \frac{1}{2730}}}}$$

The convergents are

- $\frac{p_0}{q_0} = 0$
- $\frac{p_1}{q_1} = 0 + \frac{1}{1} = 1$
- $\frac{p_2}{q_2} = 0 + \cfrac{1}{1 + \frac{1}{1}} = \frac{1}{2}$
- $\frac{p_3}{q_3} = 0 + \cfrac{1}{1 + \cfrac{1}{1 + \frac{1}{1}}} = \frac{2}{3}$
- $\frac{p_4}{q_4} = 0 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{1}{1 + \frac{1}{2730}}}} = \frac{5461}{8192}$

Example

$$CF_k(x) = [a_0, a_1, \ldots a_k] = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\ldots + \cfrac{1}{a_k}}}} = \frac{p_k}{q_k}$$

$$\pi \approx 3{,}1415926535\ldots \equiv [3, 7, 15, 1, 292, 1, 1, 1, 2, 1, 3, \ldots]$$

The convergents are

$$3, \frac{22}{7}, \frac{333}{106}, \frac{355}{113}, \frac{103993}{33102}, \ldots$$

- $\frac{p_0}{q_0} = 3$
- $\frac{p_1}{q_1} = 3 + \frac{1}{7} = \frac{22}{7} = 3.14285714\mathbf{28}$
- $\frac{p_2}{q_2} = \frac{333}{106} = 3.14150943\mathbf{39}$
- $\frac{p_3}{q_3} = \frac{355}{113} = 3.14159292\mathbf{03}$
- $\frac{p_4}{q_4} = \frac{103993}{3102} = 3.14159265\mathbf{30}$

Algorithm CFA

There exists an algorithm that efficiently computes the k-th convergent

Let $CF(x) = [a_0, a_1, a_2, \ldots]$

The k-th convergent of x

$$CF_k(x) = [a_0, a_1, \ldots a_k] = p_k \,/\, q_k$$

can be recursively computed as follows (proof by induction)

$$p_0 = a_0 \qquad q_0 = 1 \qquad k = 0$$
$$p_1 = 1 + a_0 a_1 \qquad q_1 = a_1 \qquad k = 1$$

$$p_k = a_k\, p_{k-1} + p_{k-2}$$
$$q_k = a_k\, q_{k-1} + q_{k-2} \qquad k \geq 2$$

p_k and q_k are increasing sequences, s.t.

$$\begin{aligned} p_k &\geq 2\, p_{k-2} \\ q_k &\geq 2\, q_{k-2} \end{aligned} \quad \Rightarrow \quad \begin{aligned} p_k &\geq 2^{\lfloor k/2 \rfloor} \\ q_k &\geq 2^{\lfloor k/2 \rfloor} \end{aligned}$$

Let $\frac{s}{r} = [a_0, a_1, \ldots, a_\ell]$

Let $p_k / q_k = [a_1, a_2, \ldots, a_k]$ be the k-th convergent, $1 \leq k \leq \ell$

If s and r (reduced to lowest terms) are n-bit integers, then

- the length ℓ of the continued fraction expansion is $O(n)$
- the continued fraction expansion and its convergents can be computed in time $O(n^3)$, or $O(n^2 \log n \log\log n)$ with fast multiplication

Proof (sketch)

Since $p_k \geq 2^{\lfloor k/2 \rfloor}$ and $q_k \geq 2^{\lfloor k/2 \rfloor}$, we must get s/r after at most $O(n)$ steps

- Indeed, $s/r = p_\ell / q_\ell$, and s and r are n-bit integers

The computation of a_k involves the division of $O(n)$ bit integers (and splittig off the integer part) and can be performed in $O(n^2)$ time

⇝ We can compute all $O(n)$ integers a_k in $O(n^3)$ time

Similarly, using the recurrence relations, we can compute all p_k and q_k in $O(n^3)$ time, too

Remarks

Given a rational number φ, the sequence of the convergents of its continued fractions expansion $CF(\varphi)$ has the following properties:

- the convergents are in their lowest terms
- with increasing k, the difference between φ and its k-th convergent gets smaller and smaller (i.e., the convergents form an approximation of φ that gets better and better)
- the denominator of the convergents are increasing

The convergent can be used to approximate the rational number φ by fractions with smaller denominator